

Chapter 6

Transitions, Transforms and Animation

INT150 Web Technologies (2/2025)

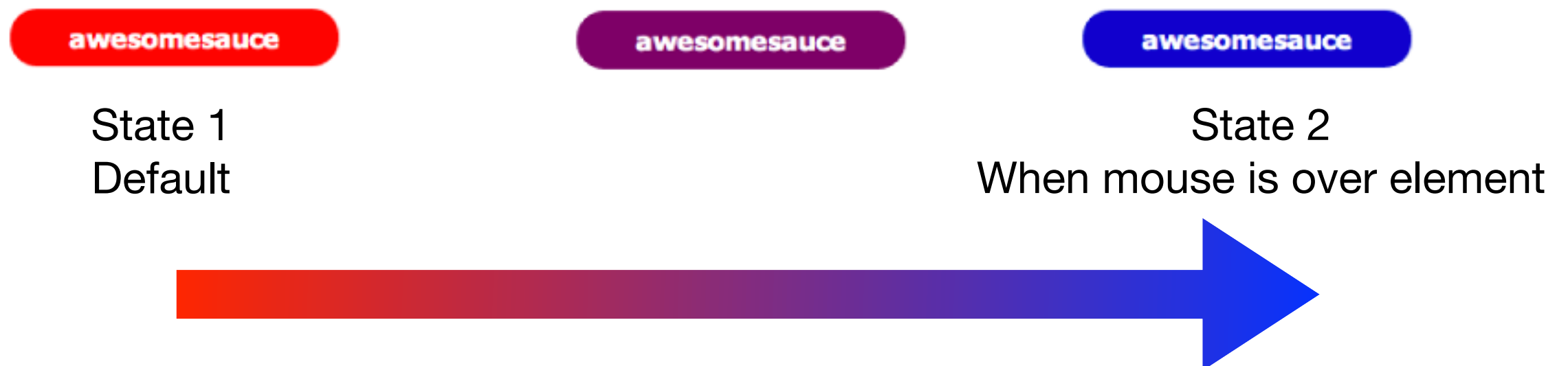
Sanit Sirisawatvatana

Topics

- Creating smooth transitions
- Moving, rotating, and scaling elements
- Combining transitions and transforms

CSS Transitions

- **CSS transitions** create a smooth change from one state to another.
- Example: Gradually change a button from red to blue (through purple) when the mouse pointer hovers over it.



Transition Properties

- transition-property
 - Which CSS property to change
- transition-duration
 - How long the transition should take in seconds (or milliseconds)
- transition-timing-function
 - The manner in which the transition accelerates
- transition-delay
 - When there should be a pause before the transition starts and how long that pause should be (in seconds)

How long should it take?

- **transition-duration** sets the amount of time it takes for the animation to complete in seconds (s) or milliseconds (ms).
- There is no correct duration but **.2s** seems to be popular transition time for UI elements.

transition-duration

Values:	time in seconds (s) or milliseconds (ms)
---------	--

Default:	0s
----------	----

Applies to:	all elements, :before and :after pseudo-elements
-------------	--

Inherits:	no
-----------	----

Specifying the property

- **transition-property** identifies the CSS property that is changing and that you want to transition smoothly.
- **Example:** foreground color, border, dimensions, front- and text-related attributes and many more.

transition-property

Values: property-name | **all** | none

Default: all

Applies to: all elements, :before and :after pseudo-elements

Inherits: no

Timing Functions

- **transition-timing-function** specifies the way the transition accelerates or decelerates over its duration.
- The default is **ease**, which starts slowly, accelerates quickly then slows down again at the end.

transition-timing-function

Values: **ease** | linear | ease-in | ease-out | ease-in-out | step-start | step-end | steps | cubic-bezier(##,##,##,##)

Default: ease

Applies to: all elements, :before and :after pseudo-elements

Inherits: no

Timing Function

linear Stays consistent from beginning to end

ease-in Starts slowly, then speed up

ease-out Starts quickly, then slow down

ease-in-out Similar to ease, but with less acceleration in the middle

cubic-bezier(##,##,##,##) Defines a curve that plots acceleration

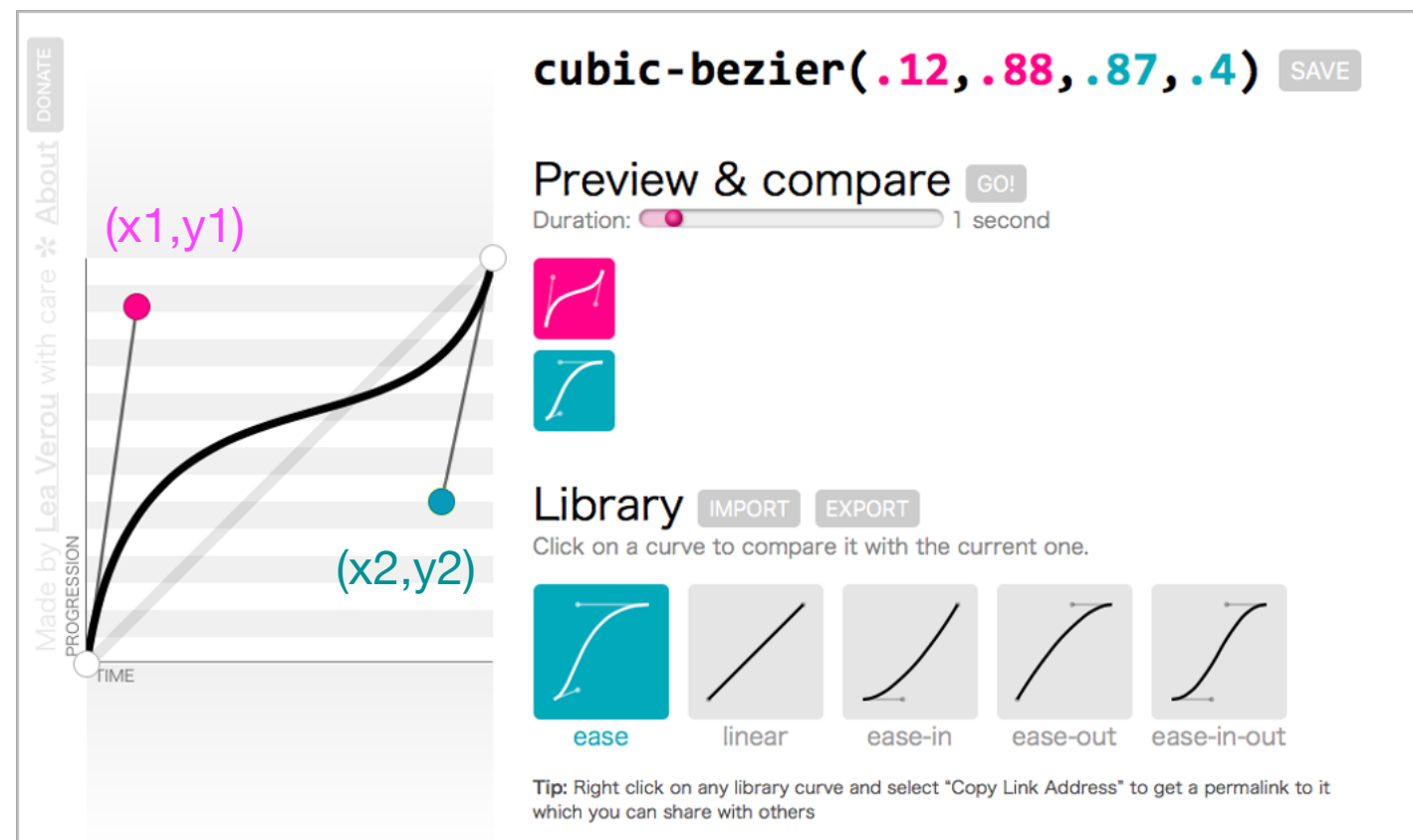
steps(##,start or end) Divides the animation into a number of steps. The start and end keywords indicate whether that transition happens at the

step-start Changes states in one step, at the beginning of the duration time (the same as steps(1,start))

step-end Change states in one step, at the end of the duration time (the same as steps(1, end))

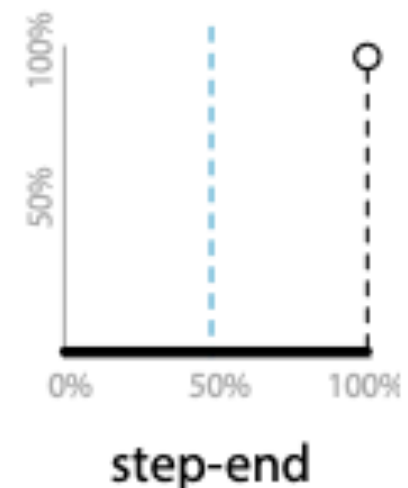
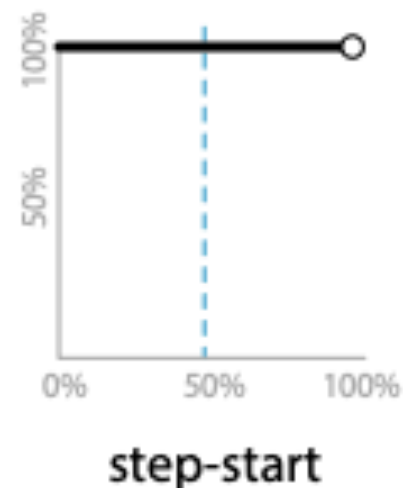
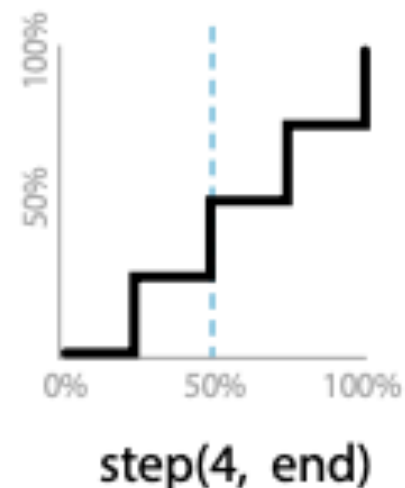
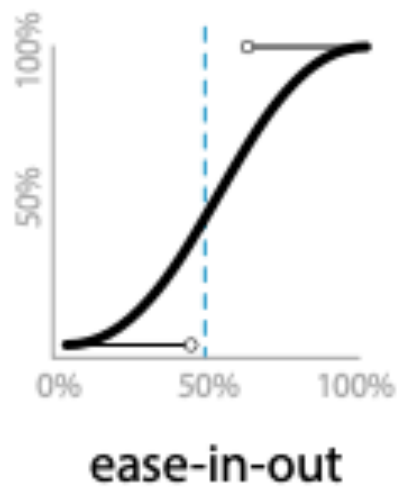
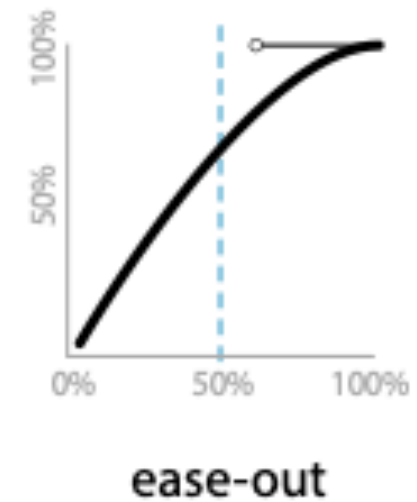
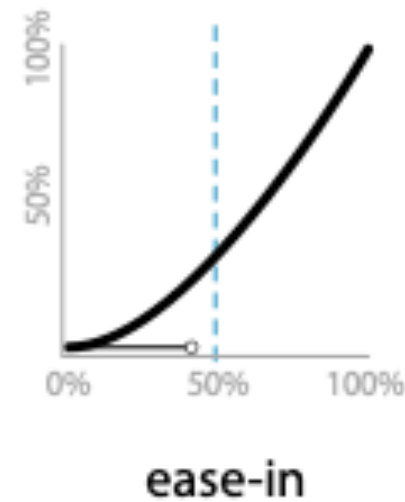
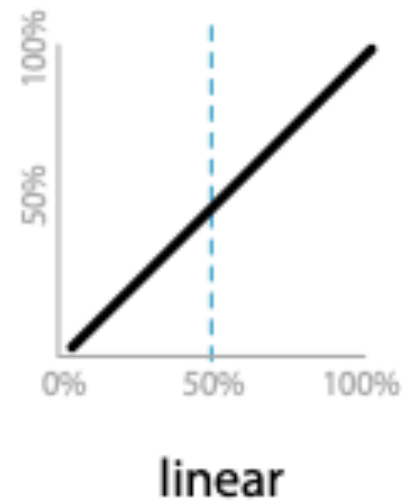
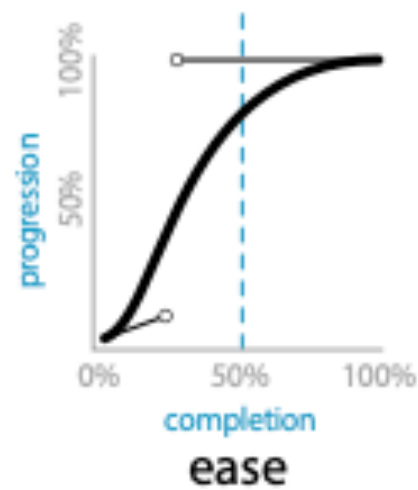
Cubic Bezier Curves

- Acceleration can be plotted using [Bezier curve](#).
- The steep parts of the curve indicate a fast rate of change, and the flat parts indicate a slow rate of change.
- The curve is defined by the x, y coordinates of "handler" that control the curve



[\[Link\]](#)

Curves for transition-timing-function



Transition Delay

- **transition-delay** delays the start of the animation by a specified amount of time.

transition-delay

Values:	time
---------	------

Default:	0s
----------	----

Applies to:	all elements, :before and :after pseudo-elements
-------------	--

Inherits:	no
-----------	----

The transition property (shorthand)

- The shorthand transition property combines all of the transition properties into one declaration.
- The value for each of the transition-* are listed, separated by character spaces.
- The order isn't important as long as the duration (which is required) appears before delay (which is optional).

transition

Values:	property duration [timing-function] [delay]
---------	---

Default:	no
----------	----

Applies to:	all elements, :before and :after pseudo-elements
-------------	--

Inherits:	no
-----------	----

Applying multiple transitions

- To apply multiple transitions is to list all the values for each property separated by comma.
- The values are matched up according to their positions in the list.
- Or lie up values for the shorthand **transition** property as well.

```
.smooth {  
  ...  
  transition-property: background-color, color, letter-spacing;  
  transition-duration: 0.3s, 2s, 0.3s;  
  transition-timing-function: ease-out, ease-in, ease-out;  
}  
.smooth:hover, .smooth:focus {  
  background-color: red;  
  letter-spacing: 3px;  
  color: black;  
}
```

```
.smooth {  
  ...  
  transition: background-color 0.3s ease-out,  
             color 2s ease-in,  
             letter-spacing 0.3s ease-out;  
}
```

A transaction for All occasions

- For cases when you want the same duration, timing function, and delay to apply to all occasions that might occur on an element, use the **all** value for **transition-property**.
- **Example:** Any property that might change for `.smooth` element should last `.2s` and animate via the `ease-in-out` function.

```
.smooth {  
  ...  
  transition: all 0.2s ease-in-out;  
}
```

Steps for creating transition

1. Set initial state: The default look of the element

```
.dividend-card {  
  background-color: #ffffff;  
  transform: scale(1);  
  padding: 20px;  
  border: 1px solid #ddd;  
}
```

2. Define transition: The "clock" that tells the browser how to animate.

```
.dividend-card {  
  /* Syntax: [property] [duration] [timing-function] */  
  transition: all 0.3s ease-in-out;  
}
```

3. Set the target state: The look of the element after an interaction (e.g., hover, focus)

```
/* When user hovers over the stock card */  
.dividend-card:hover {  
  background-color: #f9f9f9;  
  transform: scale(1.05); /* Slightly enlarge */  
  border-color: #2185d0; /* Semantic UI Blue */  
}
```


CSS Transforms

- CSS3 Transforms Module provides a way to rotate, relocate, resize and skew HTML elements in both two- and three-dimensional space.
- The transform change how an element displays, but it is not motion- or time-based.
- However, you can animate from one transform state to another using transitions or keyframe animation.

transform property

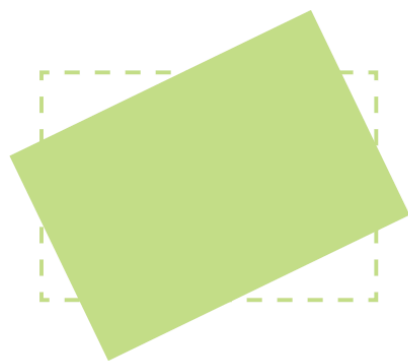
transform

Values: rotate() | rotateX() | rotateY() | rotateZ() | rotate3d() | translate() | translateX() | translateY() | scale() | scaleX() | scaleY() | skew() | skewX() | skewY() | [none](#)

Default: none

Applies to: transformable elements

Inherits: no



[rotate\(\)](#)



[translate\(\)](#)



[scale\(\)](#)



[skew\(\)](#)

Transform Behavior

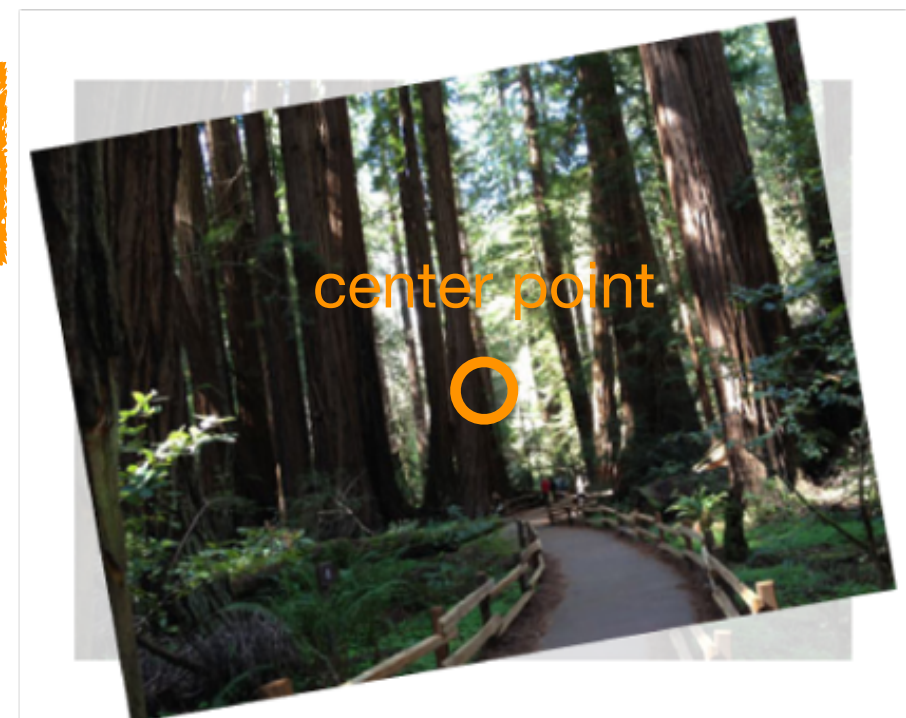
- When you apply transform to an element, it only affects the visual appearance, not the layout structure.
- The element's box stay in its original position, and the surrounding is not affected.
- It's like the browser "picks up" the pixels of the element, transforms them, and paints them back in place -- without moving the actual box.
- This is similar to "`position: relative;`" where the element move visually but still holds it original space in the layout.

Transforming the Angle (rotate)

- Use the `rotate ( )` function as the value of transform to rotate the element at given angle:
- The image rotates around its `center point` (default: 50% 50%). To change the rotate point to other points with the `transform-origin` property.

+angle clockwise
-angle counter-clockwise

```
img {  
  width: 400px;  
  height: 300px;  
  transform: rotate(-10deg);  
}
```



transform: rotate(-10deg);

Transform Origin

transform-origin

Values: percentage | length | left | center | right | top | bottom

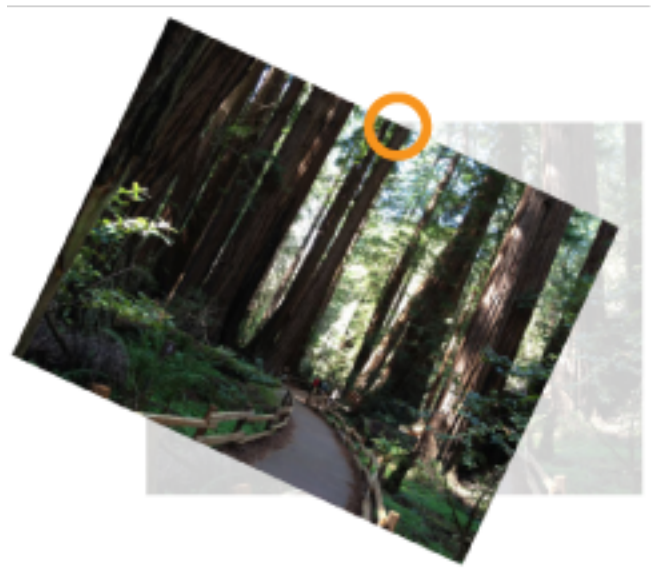
Default: 50% 50% (center point)

Applies to: transformable elements

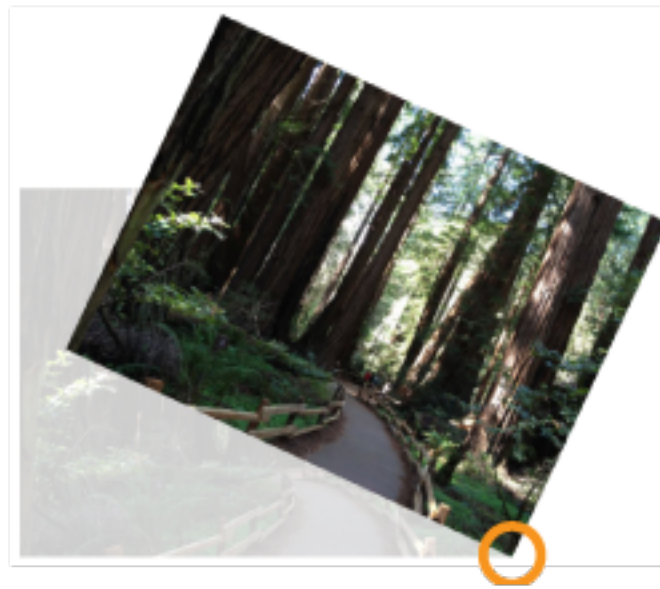
Inherits: no

- Two values (length or percentage)
 - The first value is the **horizontal offset** (x)
 - The second is the **vertical offset** (y)
- One value will be used for both horizontal and vertical offset

transform-origin



`transform-origin: center top;`



`transform-origin: 100% 100%;`



`transform-origin: 400px 0;`

`100% 0;`

Transforming the position (translate)

- Use the `translate()` function as the new of transform to render an element at a new location.
- The value are an x-offset and a y-offset. When you provide one value, it's used for both axes.



translateX, translateY

- The `translateX()` allows to move an element on horizontal axis.
- The `translateY()` is for moving along the vertical axis.
- The `translate()` combines both x and y value.
- Provide **length values** in any of CSS units or a **percentage value**.
- **Percentages** are calculated on the **width of the bounding box** (from border edge to border edge).
- You can provide positive or negative values.

Transforming Size (scale)

- Use the `scaleX()`, `scaleY()`, or `scale()` function to change the size at which an element renders.
- The value is a **unitless number** that specifies a size of ratio.
- The `scale()` shorthand provides **x-offset** and **y-offset** value (providing one value applies to both axes).



`transform: scale(1.25);`



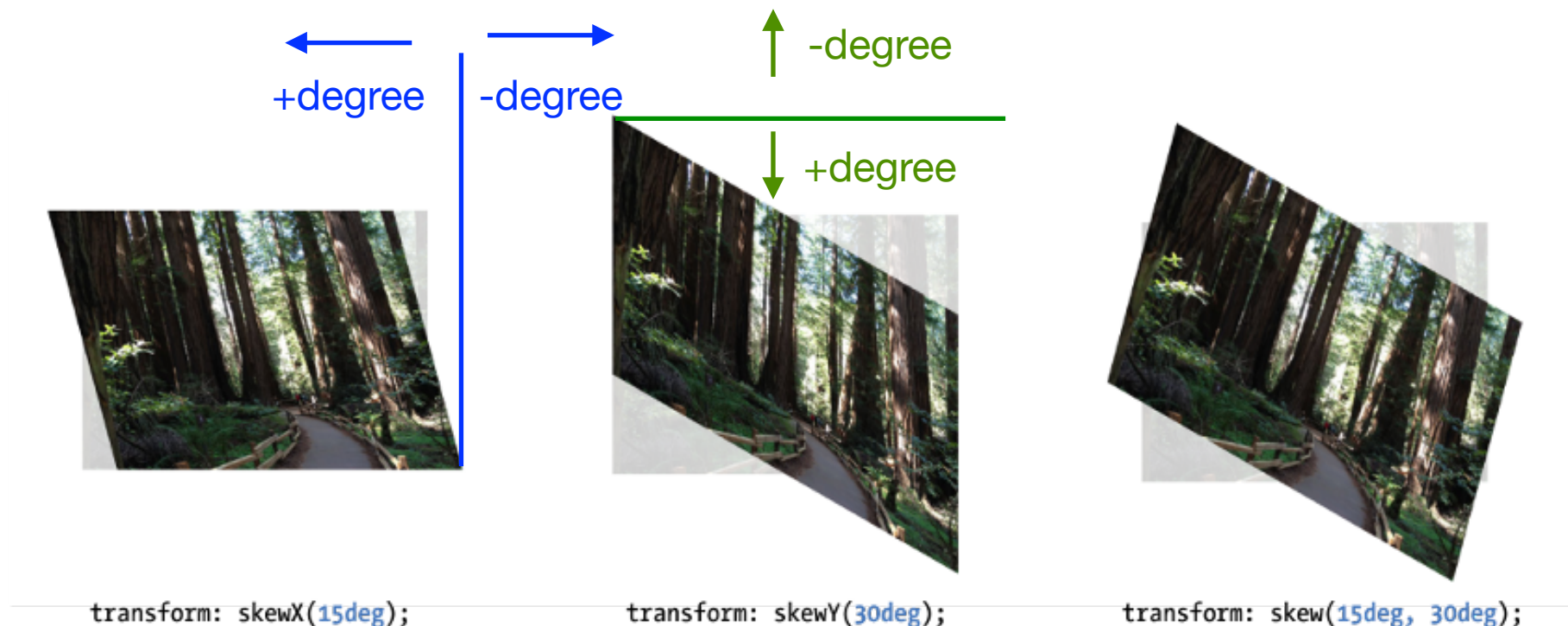
`transform: scale(.75);`



`transform: scale(1.5, .5);`

Transforming slant (skew)

- Use the `skewX()`, `skewY()`, `skew()` function to change the angle of the horizontal or vertical axes (or both).
- The value is the number of degrees the angle should be.
- The `skew()` shorthand provides `x-offset` and `y-offset` values (providing one value applies it to the x-axis only).



Applying Multiple Transforms

- You can apply more than one transform type in declaration:

```
img:hover, img:focus {  
    transform: scale(1.5) rotate(-5deg) translate(50px, 30px);  
}
```

- They're applied in the order in which they're listed. Order matters in the final result.
- If you apply a transform on an element in a different state (for example, :hover), you need to repeat all transforms already applied to the element or they will be overwritten.

Applying Multiple Transforms

```
a {  
  transform: rotate(45deg);  
}  
a:hover {  
  transform: scale(1.25); /* rotate on a element would be lost */  
}
```

- To achieve both the rotation and the scale, provide both transform values:

```
a:hover {  
  transform: rotate(45deg) scale(1.25); /* rotates and scales */  
}
```

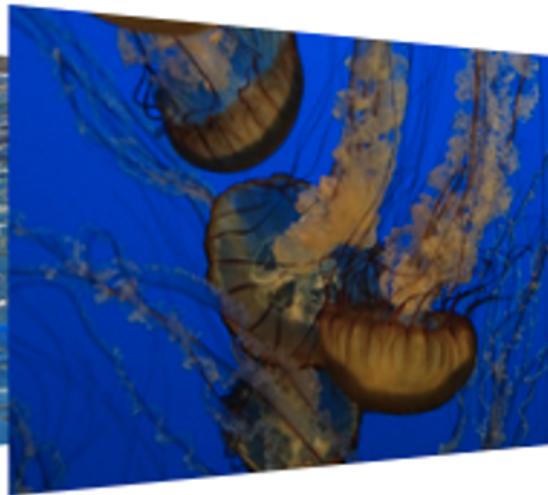
Smoothing Out Transformations

- Smooth out a transform using the transition properly.
- Example: Make an element appear to rotate smoothly where the mouse moves over it or when it's in focus

```
a:hover img.twist, a:focus img.twist {  
    transform: rotate(-5deg);  
}  
img.twist {  
    transition-property: transform;  
    transition-duration: .3s;  
}
```

3-D Transforms

- You can apply [perspective](#) to element box to make them appear as though they're in 3-D space



3-D Transform

- Apply the `perspective` property to the containing element (the lower the value, the more extreme the perspective)

```
ul {  
    ...  
    perspective: 600 ;  
}
```

- Apply one of the 3-D transform functions to each child element:

```
li {  
    ...  
    transform: rotateX(45deg) ;  
}
```

Intro to Keyframe Animation



- **Keyframe** animation enables you to create transitions between a series of states (keyframes):

1. Establish the keyframes with `@keyframes` rule:

```
@keyframes animation-name {  
  keyframe { property: value; }  
  /* additional keyframes */  
}
```

2. Apply animation properties to the element(s) that will be animated.

@keyframes rainbow

- **Keyframe** establish colors at each point in the animation and give the sequence a name "rainbow":

```
@keyframes rainbow {  
  0% { background-color: red; }  
  20% { background-color: orange; }  
  40% { background-color: yellow; }  
  60% { background-color: green; }  
  80% { background-color: blue; }  
  100% { background-color: purple; }  
}
```

- The animation properties are applied to the animated element (including which keyframe sequence to use):

```
#magic {  
  ...  
  animation-name: rainbow;  
  animation-duration: 5s;  
  animation-timing-function: linear;  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
}  
OR  
  
#magic {  
  animation: rainbow 5s linear infinite alternate ;  
}
```


Animation properties

- **animation-name** (Required)
 - Which animation to use
- **animation-duration** (Required)
 - How long it should take
- **animation-timing-function**
 - This property use the same timing function keyword in CSS Transitions (**ease**, **linear**, **ease-in**, **ease-out**,...)
- **animation-delay**
 - Whether to pause before it starts
- **animation-iteration-count**
 - How many times the animation should repeat (**number** or **infinite**)
- **animation-direction**
 - Whether the animation plays forward (**normal**), in reverse (**reverse**) or alternates back and forth starting at the beginning (**alternate**) or alternates starting from the end (**alternate-reverse**)
- **animation-fill-mode**
 - What happens with the animation before it begin and after it ends. (**none** (default), **forwards**, **backwards**, or **both**)
- **animation-play-state**
 - Whether the animation should be **running** or **paused** when it loads.

animation property (shorthand)

- The shorthand animation property:

animation: **name** **duration** timing-function delay iteration-count direction fill-mode;

- You can use the animation shorthand property to combine the values.